FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Dela Cruz, Robie D.
Section:	AN43
Professor:	Ms. Tan
Date:	November 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

COMPLETED LABORATORY TASKS

Weather Activity Advisor

This program represents weather and activity facts, then uses RBR and CBR to recommend preparations for the same situation. The sample is 32 degrees Celsius, 85% humidity, rain, 12 km/h wind, daytime, and a planned outdoor activity. It is a local teaching example.

PART 1 Knowledge Representation

Task 1A. FACTS is a Python dictionary with six facts in three explicit categories. Temperature contains temperature (32 Celsius); weather contains humidity (85%), is_raining (True), and wind_speed (12 km/h); activity_context contains is_daytime (True) and outdoor_activity (True). CATEGORIES records these groups. The input validator checks required keys, numeric ranges, and Boolean types.

PART 2 Rule Based Reasoning

Task 2A. Five IF-THEN functions define the rules: temperature > 30 activates a fan or air conditioner; rain calls for an umbrella; temperature > 30 and humidity >= 80 calls for water and heat breaks; wind >= 30 calls for securing loose outdoor items; and rain during an outdoor activity calls for moving indoors.

Task 2B. infer copies the facts into working memory, loops over every rule, records whether it fired, and collects all returned actions. This is a single-pass engine; inferred actions do not become new facts. It returns a readable no-match message if needed.

Task 2C. The sample fires four rules: hot, rain, humid heat, and outdoor rain. It recommends cooling, an umbrella, water and heat breaks, and moving the activity indoors. The wind rule does not fire because 12 is below its threshold of 30 km/h.

PART 3 Case Based Reasoning

Task 3A. CASE_BASE stores three past problem dictionaries and matching solutions. Case 1 is warm, humid, rainy daytime weather with an outdoor activity; Case 2 is mild, dry daytime weather with an outdoor activity; Case 3 is cool, rainy, windy nighttime weather with no outdoor activity. All six features and solutions appear in the complete source listing.

Retrieve. Every case is scored using all six features. Boolean matches score 1 and mismatches 0. Numeric similarity is max(0, 1 - absolute difference / range), using ranges 50 Celsius, 100 humidity percentage points, and 100 km/h. The six scores are averaged equally; the earlier case wins a tie.

Tasks 3C to 3E. Reuse returns the best case solution. Revise accepts a manual replacement or leaves the proposal unchanged. Retain appends a separate copy of the new problem and final solution. Retention is in memory; --save-cases optionally exports the case base as JSON. Each new program run starts with three seed cases.

PART 4 Comparative Reflection

RBR was easier to implement in this solution because each condition maps directly to an action. CBR also requires storing past cases, defining similarity, retrieving a match, and supporting revision and retention. CBR is more flexible for unstructured or unfamiliar problems when relevant past cases exist: a partial match provides a starting solution that a person can adapt. This still depends on choosing suitable problem features. For continuously changing experience, CBR can be more suitable because verified new cases can be retained without rewriting every rule. Obsolete cases and similarity weights still need review. RBR remains useful when stable, explicit policies must be enforced while input values change. Real-world RBR applications include eligibility checks and equipment alarms; CBR applications include technical support and adapting previous repair plans to similar faults.

Observed Results

For the same input, RBR returns four independent actions with a rule trace. CBR scores Cases 1, 2, and 3 at 98.50%, 76.67%, and 55.50%, respectively. Case 1 wins and recommends an umbrella, drinking water, and moving the activity indoors. Without a replacement, Revise keeps this advice and Retain grows the case base from 3 to 4 cases.

The Case 1 calculation is (0.98 + 0.95 + 0.98 + 1 + 1 + 1) / 6 = 0.985. Similarity expresses feature resemblance, not a probability of correctness. The implementation supports manual review; it does not automatically verify whether a recommendation worked.

Manual Revision Demonstration

Running with --revision "Move the activity to an indoor venue." replaces the retrieved solution and retains that revised text. The accompanying output capture shows the changed recommendation and the case count increasing to four. Interactive revision is also available through --interactive; Enter keeps the proposal.

Execution and Verification

Run: python ta4_reasoning.py
RBR only: python ta4_reasoning.py --mode rbr
CBR only: python ta4_reasoning.py --mode cbr
Tests: python -m unittest discover -s tests -v

The program requires Python 3.10 or later and uses only the standard library. All 15 local tests passed, including rule boundaries, retrieval, revision, retention, invalid input, and case export. The repository includes the README, example JSON, actual output text, output images, tests, and a GitHub Actions workflow.

The following pages contain the complete laboratory source file ta4_reasoning.py and captures of actual RBR, CBR, and manually revised CBR runs. Both methods use the same six input facts.

COMPLETE PYTHON CODE 1 of 4

ta4_reasoning.py | lines 1 to 54

```
"""TA4: knowledge representation, RBR, and the complete CBR 4R cycle.

Run: python ta4_reasoning.py
Optional: --mode rbr|cbr, --facts examples/weather.json, --interactive,
          --revision "Updated solution", --save-cases retained_cases.json
Uses only the Python standard library (Python 3.10 or later).
"""

import argparse
from copy import deepcopy
import json
from pathlib import Path


# PART 1: six facts, organized into three explicit knowledge categories.
FACTS = {
    "temperature": 32, "humidity": 85, "is_raining": True,
    "wind_speed": 12, "is_daytime": True, "outdoor_activity": True,
}
CATEGORIES = {
    "temperature": ("temperature",),
    "weather": ("humidity", "is_raining", "wind_speed"),
    "activity_context": ("is_daytime", "outdoor_activity"),
}
# Numeric ranges normalize distance; units are Celsius, percent, and km/h.
RANGES = {"temperature": 50, "humidity": 100, "wind_speed": 100}
LIMITS = {"temperature": (-20, 50), "humidity": (0, 100),
          "wind_speed": (0, 200)}
CASE_BASE = [
    {"problem": {"temperature": 31, "humidity": 80,
                 "is_raining": True, "wind_speed": 10,
                 "is_daytime": True, "outdoor_activity": True},
     "solution": "Bring an umbrella, drink water, and move activity indoors."},
    {"problem": {"temperature": 27, "humidity": 55,
                 "is_raining": False, "wind_speed": 12,
                 "is_daytime": True, "outdoor_activity": True},
     "solution": "Proceed outdoors and carry drinking water."},
    {"problem": {"temperature": 20, "humidity": 70,
                 "is_raining": True, "wind_speed": 40,
                 "is_daytime": False, "outdoor_activity": False},
     "solution": "Stay indoors and secure loose outdoor items."},
]


def validate_facts(facts):
    """Reject incomplete facts, wrong types, and out-of-range numbers."""
    if not isinstance(facts, dict) or set(facts) != set(FACTS):
        raise ValueError("Facts must contain exactly the six sample keys.")
    for key, (low, high) in LIMITS.items():
        value = facts[key]
        if type(value) not in (int, float) or not low <= value <= high:
            raise ValueError(f"{key} must be a number from {low} to {high}.")
    for key in set(FACTS) - set(LIMITS):
        if type(facts[key]) is not bool:
```

COMPLETE PYTHON CODE 2 of 4

ta4_reasoning.py | lines 55 to 108

```
            raise ValueError(f"{key} must be true or false.")


# PART 2: each function represents one explicit IF-THEN rule.
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on a fan or air conditioner."


def rule_rain(facts):
    if facts["is_raining"]:
        return "Bring an umbrella."


def rule_humid_heat(facts):
    if facts["temperature"] > 30 and facts["humidity"] >= 80:
        return "Drink water and take breaks from the heat."


def rule_wind(facts):
    if facts["wind_speed"] >= 30:
        return "Secure loose outdoor items."


def rule_outdoor_rain(facts):
    if facts["outdoor_activity"] and facts["is_raining"]:
        return "Move the outdoor activity indoors."


RULES = (rule_hot, rule_rain, rule_humid_heat,
         rule_wind, rule_outdoor_rain)


def infer(facts):
    """Working memory is a copy; evaluate EVERY rule, collecting all actions."""
    validate_facts(facts)
    working_memory = deepcopy(facts)
    trace, actions = [], []
    for rule in RULES:
        action = rule(working_memory)
        trace.append((rule.__name__, bool(action)))
        if action:
            actions.append(action)
    return trace, actions


# PART 3: Retrieve -> Reuse -> Revise -> Retain.
def similarity(problem, past_problem):
    """Average six scores: normalized numeric closeness or Boolean equality."""
    validate_facts(problem)
    validate_facts(past_problem)
    total = 0.0
    for key in FACTS:
        if key in RANGES:
```

COMPLETE PYTHON CODE 3 of 4

ta4_reasoning.py | lines 109 to 162

```
            distance = abs(problem[key] - past_problem[key])
            total += max(0.0, 1.0 - distance / RANGES[key])
        else:
            total += float(problem[key] == past_problem[key])
    return total / len(FACTS)


def retrieve(problem, case_base):
    """Score every case. The earlier case wins an exact similarity tie."""
    if not case_base:
        raise ValueError("Retrieval needs at least one past case.")
    scores = [similarity(problem, case["problem"]) for case in case_base]
    best = max(range(len(scores)), key=scores.__getitem__)
    return best, scores


def reuse(case):
    """Use the retrieved solution as the initial proposal."""
    return case["solution"]


def revise(solution, replacement=None):
    """A blank revision keeps the original; nonblank text replaces it."""
    return replacement.strip() if replacement and replacement.strip() else solution


def retain(problem, solution, case_base):
    """Store a separate copy so later edits cannot change past experience."""
    validate_facts(problem)
    if not isinstance(solution, str) or not solution.strip():
        raise ValueError("A retained solution must be nonblank text.")
    case_base.append({"problem": deepcopy(problem), "solution": solution.strip()})


def main(argv=None):
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--mode", choices=("both", "rbr", "cbr"), default="both")
    parser.add_argument("--facts", type=Path, help="JSON with all six input facts")
    revision = parser.add_mutually_exclusive_group()
    revision.add_argument("--interactive", action="store_true")
    revision.add_argument("--revision", help="Manual replacement for the solution")
    parser.add_argument("--save-cases", type=Path, help="Save final case base as JSON")
    args = parser.parse_args(argv)
    if args.mode == "rbr" and (args.save_cases or args.interactive
                               or args.revision is not None):
        parser.error("Revision and case export require CBR mode.")
    facts = deepcopy(FACTS)
    try:
        if args.facts:
            facts = json.loads(args.facts.read_text(encoding="utf-8"))
        validate_facts(facts)
    except (OSError, ValueError) as error:
        parser.error(str(error))
```

COMPLETE PYTHON CODE 4 of 4

ta4_reasoning.py | lines 163 to 214

```
    print("TA4 - KNOWLEDGE REPRESENTATION AND REASONING")
    print("Use case: planning activities in changing weather")
    print("\nINPUT - Knowledge Base (6 facts / 3 categories)")
    for category, keys in CATEGORIES.items():
        print(f"  {category}:")
        for key in keys:
            print(f"    {key}: {facts[key]}")

    if args.mode in ("both", "rbr"):
        trace, actions = infer(facts)
        print("\nPROCESS - Rule-Based Reasoning")
        for name, fired in trace:
            print(f"  {name}: {'FIRED' if fired else 'not fired'}")
        print("\nOUTPUT - Rule-Based Reasoning Actions:")
        for action in actions or ["No rule matched the current facts."]:
            print(f"- {action}")

    if args.mode in ("both", "cbr"):
        case_base = deepcopy(CASE_BASE)
        best, scores = retrieve(facts, case_base)
        print("\nPROCESS - Case-Based Reasoning (4R cycle)")
        print("1. RETRIEVE - Similarity of each past case:")
        for index, score in enumerate(scores, start=1):
            print(f"  Case {index}: {score:.2%}")
        print(f"  Selected: Case {best + 1}")
        proposal = reuse(case_base[best])
        print(f"2. REUSE - {proposal}")
        replacement = args.revision
        if args.interactive:
            try:
                replacement = input("Enter revision (Enter keeps solution): ")
            except EOFError:
                replacement = None
        final = revise(proposal, replacement)
        print(f"3. REVISE - {final}")
        print(f"  Solution {'unchanged' if final == proposal else 'manually revised'}.")
        before = len(case_base)
        retain(facts, final, case_base)
        print(f"4. RETAIN - Case base: {before} -> {len(case_base)} cases")
        print(f"\nOUTPUT - CBR Recommendation:\n{final}")
        if args.save_cases:
            try:
                args.save_cases.write_text(
                    json.dumps(case_base, indent=2) + "\n", encoding="utf-8")
            except OSError as error:
                parser.error(str(error))
            print(f"Saved final case base to: {args.save_cases}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

RULE BASED REASONING OUTPUT

Four rules fire and all four inferred actions are returned. The wind rule is evaluated but does not fire.

Capture of actual Python standard output displayed locally; the corresponding text file is included in examples.

Rule-Based Reasoning Output

Actual stdout from the included Python program, displayed for capture.

```
python ta4_reasoning.py --mode rbr
```

```
TA4 - KNOWLEDGE REPRESENTATION AND REASONING
Use case: planning activities in changing weather

INPUT - Knowledge Base (6 facts / 3 categories)
  temperature:
    temperature: 32
  weather:
    humidity: 85
    is_raining: True
    wind_speed: 12
  activity_context:
    is_daytime: True
    outdoor_activity: True

PROCESS - Rule-Based Reasoning
  rule_hot: FIRED
  rule_rain: FIRED
  rule_humid_heat: FIRED
  rule_wind: not fired
  rule_outdoor_rain: FIRED

OUTPUT - Rule-Based Reasoning Actions:
- Turn on a fan or air conditioner.
- Bring an umbrella.
- Drink water and take breaks from the heat.
- Move the outdoor activity indoors.
```

CASE BASED REASONING OUTPUT

Case 1 is retrieved at 98.50% similarity. Reuse and Revise preserve its solution, and Retain grows the case base from 3 to 4 cases.

Capture of actual Python standard output displayed locally; the corresponding text file is included in examples.

Case-Based Reasoning Output

Actual stdout from the included Python program, displayed for capture.

```
python ta4_reasoning.py --mode cbr
```

```
TA4 - KNOWLEDGE REPRESENTATION AND REASONING
Use case: planning activities in changing weather

INPUT - Knowledge Base (6 facts / 3 categories)
  temperature:
    temperature: 32
  weather:
    humidity: 85
    is_raining: True
    wind_speed: 12
  activity_context:
    is_daytime: True
    outdoor_activity: True

PROCESS - Case-Based Reasoning (4R cycle)
1. RETRIEVE - Similarity of each past case:
  Case 1: 98.50%
  Case 2: 76.67%
  Case 3: 55.50%
  Selected: Case 1
2. REUSE - Bring an umbrella, drink water, and move activity indoors.
3. REVISE - Bring an umbrella, drink water, and move activity indoors.
  Solution unchanged.
4. RETAIN - Case base: 3 -> 4 cases

OUTPUT - CBR Recommendation:
Bring an umbrella, drink water, and move activity indoors.
```

CASE BASED REASONING MANUAL REVISION

The manual replacement is used as the final recommendation and retained with the new problem. The case count increases to four.

Capture of actual Python standard output displayed locally; the corresponding text file is included in examples.

Case-Based Reasoning Manual Revision

Actual stdout from the included Python program, displayed for capture.

```
python ta4_reasoning.py --mode cbr --revision "Move the activity to an indoor venue."
```

```
TA4 - KNOWLEDGE REPRESENTATION AND REASONING
Use case: planning activities in changing weather

INPUT - Knowledge Base (6 facts / 3 categories)
  temperature:
    temperature: 32
  weather:
    humidity: 85
    is_raining: True
    wind_speed: 12
  activity_context:
    is_daytime: True
    outdoor_activity: True

PROCESS - Case-Based Reasoning (4R cycle)
1. RETRIEVE - Similarity of each past case:
  Case 1: 98.50%
  Case 2: 76.67%
  Case 3: 55.50%
  Selected: Case 1
2. REUSE - Bring an umbrella, drink water, and move activity indoors.
3. REVISE - Move the activity to an indoor venue.
  Solution manually revised.
4. RETAIN - Case base: 3 -> 4 cases

OUTPUT - CBR Recommendation:
Move the activity to an indoor venue.
```